

Distributed LLM Training — Run Plan

Llama-3-8B · 4× A100 SXM 80GB · RunPod · MLflow tracking

Stage 1 — Single GPU (Peak Memory Experiment)

Goal: measure actual HBM needed at any moment to decide which GPU tier fits each configuration.

Run 1	Gradient checkpointing ON python train_single.py
Run 2	Gradient checkpointing OFF (comment out gradient_checkpointing_enable(), change run_name)
Metrics logged	samples_per_sec · steady_memory_mb · peak_memory_mb
Key formula	$\text{peak(OFF)} - \text{peak(ON)} = \text{activation memory saved by checkpointing}$ $\text{savings \%} = (\text{peak(OFF)} - \text{peak(ON)}) / \text{peak(OFF)} \times 100\%$
Decision	peak_memory_mb = minimum GPU HBM needed if peak(ON) fits a cheaper GPU → checkpointing pays off cost = 18% throughput penalty

Why peak and not steady-state:

Steady-state (after optimizer.step) only shows weights + gradients + optimizer states — activations already freed. OOMs happen during the forward→backward window. Peak memory right after backward() is the true ceiling.

Stage 2 — DDP (1, 2, 4 GPUs)

Goal: measure throughput scaling and communication overhead as GPUs increase.

Commands	torchrun --nproc_per_node=1 train_ddp.py torchrun --nproc_per_node=2 train_ddp.py torchrun --nproc_per_node=4 train_ddp.py
Batch size	Fixed at 4 across all runs (clean comparison)
Metrics logged	samples_per_sec · peak_memory_mb per rank
Key formula	$\text{scaling efficiency} = (\text{N_gpu_throughput} / (\text{N} \times \text{1gpu_throughput})) \times 100\%$ target: 80–92% at 4 GPUs on NVLink
Memory expectation	Peak per rank = same as single GPU DDP does not shard — full model on every rank
Throughput expectation	~1.8× at 2 GPUs, ~3.4× at 4 GPUs (not linear — all-reduce communication overhead)

DDP answers: how much throughput do you gain by adding GPUs? Memory problem unchanged.

Stage 3 — FSDP (4 GPUs, no gradient checkpointing)

Goal: show memory savings from sharding. Compare peak memory to DDP.

Commands	torchrun --nproc_per_node=4 train_fsdp.py (batch=4) torchrun --nproc_per_node=4 train_fsdp.py (batch=16) torchrun --nproc_per_node=2 train_fsdp.py (batch=16)
-----------------	---

Gradient checkpointing	OFF — sharding alone should be enough on A100 80GB
Metrics logged	<code>samples_per_sec · peak_memory_mb · steady_memory_mb</code>
Memory expectation	Peak per rank dramatically lower than DDP FSDP shards weights + gradients + optimizer states across 4 GPUs
Throughput story	FSDP 4GPU batch=4 → lower than DDP (communication overhead) FSDP 4GPU batch=16 → back to ~DDP level (bigger batch amortizes cost) FSDP 2GPU batch=16 → can 2 GPUs match DDP 4 GPU throughput?
Key insight	DDP is memory-constrained on batch size. FSDP frees memory → bigger batch → recovers throughput lost to communication. FSDP 2 GPUs ≈ DDP 4 GPUs in throughput = same job, half the GPU cost.

Stage 4 — Ray Train (4 GPUs)

Goal: compare setup complexity and throughput vs torchrun DDP.

Command	<code>python train_ray.py</code>
Memory expectation	Same as DDP — Ray Train wraps DDP, same memory math
Throughput expectation	Same as DDP — same underlying communication
What changes	No manual <code>dist.init_process_group</code> No <code>MASTER_ADDR/PORT</code> setup Native HPO integration via Ray Tune
Decision	Same numbers, simpler setup. Worth it for large clusters where multi-node coordination is the hard part.

Expected Results Summary

Stage	GPUs	Batch	Peak Mem/rank	Throughput	Key finding
Single GPU (ckpt ON)	1	4	TBD	4.61 s/s	Baseline
Single GPU (ckpt OFF)	1	4	TBD	5.45 s/s	peak diff = activation cost
DDP	2	4	~same as single	TBD	scaling efficiency at 2 GPUs
DDP	4	4	~same as single	TBD	scaling efficiency at 4 GPUs
FSDP	4	4	TBD (much lower)	TBD	apples-to-apples vs DDP
FSDP	4	16	TBD (much lower)	TBD	recover throughput with freed memory
FSDP	2	16	TBD (much lower)	TBD	2 GPUs ≈ DDP 4 GPUs?
Ray Train	4	4	~same as DDP	TBD	setup simplicity vs torchrun

Tracking Results — compare_runs.py

After each stage, run `compare_runs.py` on the pod to auto-calculate scaling efficiency and peak memory savings. Reads all runs from MLflow — runs not yet completed show TBD.

Run on pod	<code>python compare_runs.py</code> (MLflow server must be running at localhost:8080)
-------------------	--

Memory table	$\text{peak_memory_mb} \cdot \text{steady_memory_mb} \cdot \text{activation_memory per run}$ $\text{checkpointing savings \%} = (\text{peak_OFF} - \text{peak_ON}) / \text{peak_OFF} \times 100\%$
Scaling table	DDP and FSDP only $\text{expected} = \text{single_gpu_throughput} \times \text{world_size}$ $\text{actual multiplier} = \text{run_throughput} / \text{single_gpu_throughput}$ $\text{efficiency \%} = (\text{actual} / \text{expected}) \times 100\%$
Partial runs OK	Script works at any stage — missing runs print TBD Run after single GPU → after DDP → after FSDP to see table fill progressively

GPU sizing decision from peak memory:

$\text{peak}(\text{checkpointing OFF})$ = minimum GPU HBM without tricks. $\text{peak}(\text{checkpointing ON})$ = minimum GPU HBM with checkpointing. If the cheaper GPU tier fits $\text{peak}(\text{ON})$, checkpointing pays off — cost is 18% throughput.